

# COMSATS University Islamabad – Lahore Campus

Department of Computer Science | CSC241 Object Oriented Programming (Java)

## ASSESSMENT

### Theory Assignment 1

## PERSONAL ACCESS CODE

**BAI-TA1-382809B9D346**

## STUDENT CONDITION

**Privacy First (Last Digit = 1)**

## REGISTRATION NO.

**CIIT/SP26-BAI-051/LHR**

## INSTRUCTOR

**Dr. Muhammad Shahid Bhatti**

## DUE DATE

**26 September 2026**

**AI Disclosure:** This assignment was prepared with the assistance of Claude (Anthropic) and Gemini (Google) as writing, structuring, and formatting tools. Design decisions, class models, state transition logic, and reasoning were developed collaboratively and reviewed by the author. The author takes full responsibility for every claim, design choice, and conclusion presented herein.

## TASK 1 – STAKEHOLDER AND REQUIREMENT ANALYSIS

### 1.1 Stakeholders and Needs

- **Student:** Needs to check real-time room availability for individual or group work while ensuring that their own personal information and booking reasons remain private from other students.
- **Faculty Member:** Needs priority booking for scheduled teaching and urgent academic activities, with space availability clearly visible without exposing other users' personal details.
- **Administrator:** Needs aggregate utilization reports to monitor and manage overall campus space efficiency without requiring access to individual user identities.
- **Maintenance Staff:** Needs the authority to block unsafe or broken rooms immediately and record maintenance reasons independently of who currently holds a booking.
- **Security Staff:** Needs a traceable access log for incident investigation and security auditing, strictly restricted from general system users.
- **Disability Support Staff:** Needs accessible rooms to remain reserved for students requiring specific accommodations without exposing private disability information to the public.

### 1.2 Genuine Requirement Conflicts and Assumptions

- **Conflict 1 – Fairness vs. Faculty Priority:** Students expect first-come-first-served access, while faculty members require guaranteed slots for teaching and urgent academic events.  
*Assumption/Decision:* Timetabled faculty reservations take precedence over student bookings made less than 72 hours in advance; otherwise, first-come-first-served applies, enforced centrally in `AllocationPolicy`.
- **Conflict 2 – High Utilization vs. Safety Blocking:** Administrators aim for maximum room usage, whereas maintenance staff must instantly close rooms for safety issues regardless of existing bookings.  
*Assumption/Decision:* A maintenance block instantly overrides confirmed reservations. Relocation is attempted automatically, and if no alternative space is found within 30 minutes, the reservation is cancelled with a generic notification sent to the affected user.
- **Conflict 3 – Privacy vs. Transparency (Student Condition: Privacy First):** Ordinary users need to know whether a room is occupied, but not who booked it or why, whereas security staff require detailed logs and administrators need capacity figures.  
*Assumption/Decision:* The system separates data visibility into a public busy/free view (`AvailabilityView`) with no identity or purpose fields, and a restricted audit view gated by an explicit permission check (`VIEW_AUDIT_DATA`). Administrative reports receive aggregated counts only.

## TASK 2 – OBJECT ORIENTED CONCEPTUAL MODEL

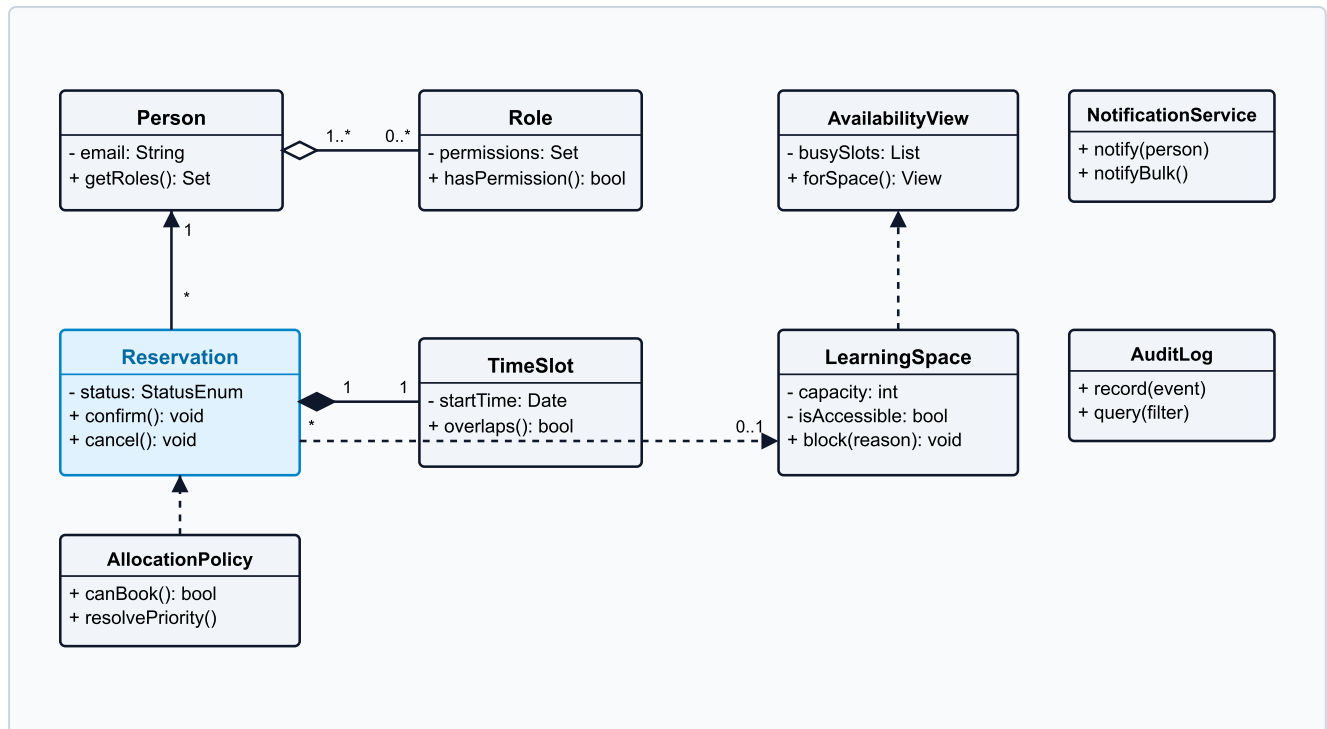
### 2.1 Proposed Classes

| Class                      | Responsibility                                                                        | Key Attributes / Operations                                                    |
|----------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Person</b>              | Represents any system user and manages their identity and assigned roles.             | roles: Set<Role>, email<br>getRoles(), addRole(r)                              |
| <b>Role</b>                | Represents a user role type and its associated permission set, modifiable at runtime. | roleType, permissions: Set<Permission><br>hasPermission(p), isActive(date)     |
| <b>LearningSpace</b>       | Represents a bookable physical facility and its safety/availability status.           | capacity, isAccessible<br>isAvailable(slot), block(reason)                     |
| <b>Equipment</b>           | Represents a scarce or movable resource allocated independently of the space.         | isMovable, isAvailable<br>allocateTo(r), release()                             |
| <b>Reservation</b>         | Owns the lifecycle and state transitions of a booking request from draft to closure.  | status, timeSlot<br>submit(), confirm(), cancel()                              |
| <b>TimeSlot</b>            | Represents a bounded time duration and checks temporal overlaps.                      | startTime, endTime<br>overlapsWith(other), duration()                          |
| <b>AllocationPolicy</b>    | Centralises business logic: priority rules, overlap checks, and permission gating.    | priorityRules, maxRecurringSlots<br>canBook(p, s, t),<br>resolvePriority()     |
| <b>AvailabilityView</b>    | Produces the privacy-filtered public representation of space bookings.                | spaceId, busySlots: List<TimeSlot><br>forSpace(s),<br>containsIdentity()=false |
| <b>AuditLog</b>            | Stores an immutable, pseudonymous record of security and operational events.          | logId, actorId (pseudonymous)<br>record(event), query(filter)                  |
| <b>NotificationService</b> | Handles secure, targeted messaging to individual users.                               | channel<br>notify(person, msg), notifyBulk()                                   |

### 2.2 Class Relationships & Multiplicities

| Relationship                          | Type        | Multiplicity | Rationale                                                                         |
|---------------------------------------|-------------|--------------|-----------------------------------------------------------------------------------|
| <b>Person – Role</b>                  | Aggregation | 1..* → 0..*  | A person holds zero or more roles at once. Open diamond on Person.                |
| <b>Reservation – Person</b>           | Association | * → 1        | Many reservations can be created by one person. Directed line to Person.          |
| <b>Reservation – LearningSpace</b>    | Association | * → 0..1     | A reservation targets at most one space. Directed line to LearningSpace.          |
| <b>Reservation – TimeSlot</b>         | Composition | 1 → 1        | A TimeSlot is uniquely owned and managed by a single reservation. Filled diamond. |
| <b>Reservation – Equipment</b>        | Association | * → 0..*     | A reservation may request multiple equipment items.                               |
| <b>AllocationPolicy – Reservation</b> | Dependency  | 1 → *        | AllocationPolicy validates reservation rules prior to confirmation. Dashed arrow. |

### 2.2.1 Conceptual UML Class Diagram



### 2.3 Responsibility Separation

To avoid a monolithic single controller, system operations are divided across distinct domain classes:

- **Person** manages only personal identity and role membership.
- **Reservation** owns its lifecycle states (DRAFT, PENDING, CONFIRMED, CANCELLED) but delegates authorization and rule evaluation to **AllocationPolicy**.
- **LearningSpace** maintains its physical capacity and maintenance status without storing user identity.
- **AvailabilityView** acts as a read-only projection that strips sensitive identity fields at the data query layer.
- **AuditLog** and **NotificationService** act as decoupled utility services providing logging and messaging.

## TASK 3 – ABSTRACTION, ENCAPSULATION AND INVARIANTS

### 3.1 Abstractions and Hidden Information

- **Reservation**: Abstracts a physical room booking into a single state machine. It hides internal state transitions and user details, which are reachable in audit trails only via pseudonymous identifiers. State changes are encapsulated through `submit()`, `confirm()`, `cancel()`, and `relocate()`.
- **AvailabilityView**: Abstracts a room's calendar into a simple binary busy/free schedule. Sensitive information such as requester name, student ID, or event purpose is structurally absent from this class, preventing accidental privacy leaks.
- **AuditLog**: Abstracts security investigation logs into an append-only event stream. It encapsulates log storage, exposing only `record()` and `query(filter)` while hiding storage internals and preventing modification or deletion.

## 3.2 System Invariants

| ID    | Invariant Rule                                                                                                                           | How Encapsulation Enforces It                                                                                                       |
|-------|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| INV-1 | A public query response from AvailabilityView never contains user identity or event purpose.                                             | Attributes for identity and purpose are omitted from the class model entirely; fields cannot be accessed or populated.              |
| INV-2 | A Reservation must strictly follow valid state transitions (DRAFT → PENDING → CONFIRMED → COMPLETED / CANCELLED; CONFIRMED → SUSPENDED). | The status attribute is private and can only be updated through controlled lifecycle methods that reject invalid state transitions. |
| INV-3 | Two CONFIRMED reservations must not occupy overlapping TimeSlots for the same LearningSpace.                                             | AllocationPolicy.canBook() checks for time overlaps against existing confirmed slots before confirm() can execute.                  |
| INV-4 | AuditLog entries are immutable and store pseudonymous actor IDs rather than direct personal names.                                       | Log records are stored in a private list without setter methods; entries are added solely via record().                             |

## TASK 4 – ALTERNATIVE DESIGNS AND DECISION

### 4.1 Two Alternatives for Multi-Role Users

- **Option A – Inheritance Hierarchy:** Person serves as an abstract base class with concrete subclasses such as Student, FacultyMember, and Administrator.
- **Option B – Role-Object Composition (Chosen):** Person maintains a Set<Role> collection. Role is an abstract class with subclasses for specific roles, each holding explicit permission sets that can be modified at runtime.

### 4.2 Architectural Comparison

| Criterion                  | Inheritance (Option A)                                                            | Role-Object Composition (Option B)                                                                      |
|----------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Multiple Roles             | Requires multiple inheritance, which is unsupported in Java.                      | Handled naturally by storing multiple Role instances inside a Set<Role> collection.                     |
| Runtime Permission Changes | Inflexible; changing roles requires instantiating a new object and copying state. | Dynamic; roles can be added or removed at runtime without altering the Person instance.                 |
| Coupling & Extensibility   | High coupling; adding a new role requires extending the core class hierarchy.     | Low coupling; follows the Open/Closed Principle by adding new Role subclasses without modifying Person. |

### 4.3 Decision and Justification

**Option B (Role-Object Composition)** is selected based on three core requirements:

1. Users frequently hold concurrent roles (e.g., a student serving as a teaching assistant). Composition represents this relationship directly without complex class hierarchies.
2. Roles and permissions change mid-semester. Composition allows dynamic additions and removals of Role objects at runtime.
3. The **Privacy First** requirement requires checking specific permissions (VIEW\_AUDIT\_DATA) before exposing detailed reservation records. Iterating over Person.getRoles() cleanly supports rule enforcement.

## TASK 5 – BEHAVIOUR AND EDGE CASES

### 5.1 Reservation Lifecycle State Transitions

| Current State         | Event / Trigger             | Next State | System Action                                                  |
|-----------------------|-----------------------------|------------|----------------------------------------------------------------|
| DRAFT                 | User submits request        | PENDING    | Validates room capacity and checks slot availability.          |
| PENDING               | Approval granted            | CONFIRMED  | Locks TimeSlot, updates state, and sends notification to user. |
| PENDING               | Request rejected            | REJECTED   | Sends rejection notice with generic reason.                    |
| CONFIRMED             | Maintenance block issued    | SUSPENDED  | Marks room unavailable and initiates automated relocation.     |
| CONFIRMED             | User cancels booking        | CANCELLED  | Releases time slot and records action in AuditLog.             |
| CONFIRMED             | Slot end time reached       | COMPLETED  | Releases room allocation and archives booking record.          |
| SUSPENDED             | Maintenance block cleared   | CONFIRMED  | Restores original room allocation and notifies user.           |
| SUSPENDED / CONFIRMED | Relocation succeeded        | CONFIRMED  | Reassigns reservation to new LearningSpace and notifies user.  |
| SUSPENDED / CONFIRMED | Relocation timeout (30 min) | CANCELLED  | Cancels booking, releases resources, and logs event.           |

### 5.2 Handling System Edge Cases

- **Simultaneous Requests for the Same Space:** Both requests enter PENDING status. AllocationPolicy evaluates them sequentially using a thread lock on the time slot. The first request passing canBook( ) transitions to CONFIRMED, while the second request is rejected. Tie-breaking relies on submission timestamps.  
*Responsible Object:* AllocationPolicy enforces priority evaluation, while LearningSpace maintains slot state.
- **Maintenance Block After Confirmation:** Calling LearningSpace.block(reason) transitions all affected CONFIRMED reservations to SUSPENDED and requests relocation via AllocationPolicy. If an alternative space is found within 30 minutes, the reservation updates to the new space; otherwise, it transitions to CANCELLED.  
*Responsible Object:* LearningSpace triggers the event, AllocationPolicy finds alternatives, and Reservation manages its state transitions.
- **Emergency Cancellation or Relocation:** An emergency event triggers AllocationPolicy to process affected bookings. Notifications are dispatched individually via NotificationService. In accordance with the **Privacy First** requirement, messages sent to users contain only their own reassigned space details without mentioning other impacted users.  
*Responsible Object:* An emergency orchestrator handles batch relocation, NotificationService dispatches private alerts, and AuditLog records event references.

## TASK 6 – INDEPENDENT EXPLORATION

### 6.1 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is an access control security architecture where access permissions are assigned to specific roles rather than directly to individual user accounts (Sandhu et al., 1996). Users inherit permissions by being assigned to roles, allowing system administrators to modify user access rights by updating role definitions rather than editing individual user accounts (Ferraiolo et al., 2001).